

CS790 Assignment 3 Task 1

Title: CS790 Assignment 3 Task 1

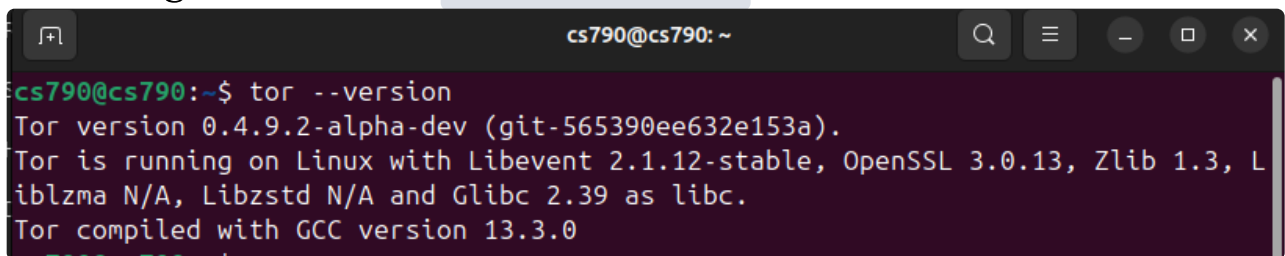
Name: Vansil Rakeshkumar Chauhan 24m0847

1. Download the Tor source code, compile it, and install it on one of your VMs.

- Installing tor using following commands

```
sudo ./autogen.sh
sudo ./config
sudo make
sudo make install
```

- Checking tor --version `tor --version`

A terminal window with a dark background and light text. The title bar shows 'cs790@cs790: ~'. The command 'tor --version' has been executed, and the output is displayed in green and white text. The output indicates the Tor version is 0.4.9.2-alpha-dev, compiled with GCC 13.3.0, and lists various system dependencies like Libevent, OpenSSL, Zlib, and Glibc.

```
cs790@cs790:~$ tor --version
Tor version 0.4.9.2-alpha-dev (git-565390ee632e153a).
Tor is running on Linux with Libevent 2.1.12-stable, OpenSSL 3.0.13, Zlib 1.3, L
iblzma N/A, Libzstd N/A and Glibc 2.39 as libc.
Tor compiled with GCC version 13.3.0
```

- Tor installed successfully.
-

2. Create a custom Tor client circuit using either the Tor control port (default is TCP 9051) or using the Tor `stem` library. The Tor circuit

should have a length of 4 hops instead of the usual 3 hops.

- Create and Edit `/usr/local/etc/tor/torrc` file for exposing the control port `9051` that allows applications to interact with the Tor process.
- Create:

```
cs790@cs790:/usr/local/etc/tor$ code torrc
cs790@cs790:/usr/local/etc/tor$ touch torrc
touch: cannot touch 'torrc': Permission denied
cs790@cs790:/usr/local/etc/tor$ sudo touch torrc
cs790@cs790:/usr/local/etc/tor$ code torrc
cs790@cs790:/usr/local/etc/tor$ sudo code torrc
sudo: code: command not found
cs790@cs790:/usr/local/etc/tor$ nano torrc
cs790@cs790:/usr/local/etc/tor$ sudo torrc
sudo: torrc: command not found
cs790@cs790:/usr/local/etc/tor$ sudo nano torrc
cs790@cs790:/usr/local/etc/tor$
```

- Edit:

```
GNU nano 7.2 torrc
ControlPort 9051
CookieAuthentication 1
```

- Run Tor using commands `tor -f /usr/local/etc/tor/torrc`

```
cs790@cs790:~$ tor -f /usr/local/etc/tor/torrc
Apr 14 16:01:17.666 [notice] Tor 0.4.9.2-alpha-dev (git-565390ee632e153a) running on Linux with Libevent 2.1.12-stable, OpenSSL
3.0.13, Zlib 1.3, Liblzma N/A, Libzstd N/A and Glibc 2.39 as libc.
Apr 14 16:01:17.666 [notice] Tor can't help you if you use it wrong! Learn how to be safe at https://support.torproject.org/faq/
staying-anonymous/
Apr 14 16:01:17.666 [notice] This version is not a stable Tor release. Expect more bugs than usual.
Apr 14 16:01:17.666 [notice] Read configuration file "/usr/local/etc/tor/torrc".
Apr 14 16:01:17.667 [notice] Opening Socks listener on 127.0.0.1:9050
Apr 14 16:01:17.667 [notice] Opened Socks listener connection (ready) on 127.0.0.1:9050
Apr 14 16:01:17.667 [notice] Opening Control listener on 127.0.0.1:9051
Apr 14 16:01:17.667 [notice] Opened Control listener connection (ready) on 127.0.0.1:9051
Apr 14 16:01:17.000 [notice] Parsing GEOIP IPv4 file /usr/local/share/tor/geoip.
Apr 14 16:01:17.000 [notice] Parsing GEOIP IPv6 file /usr/local/share/tor/geoip6.
Apr 14 16:01:17.000 [notice] Bootstrapped 0% (starting): Starting
Apr 14 16:01:18.000 [notice] Starting with guard context "default"
Apr 14 16:01:19.000 [notice] Bootstrapped 5% (conn): Connecting to a relay
Apr 14 16:01:19.000 [notice] Bootstrapped 10% (conn_done): Connected to a relay
Apr 14 16:01:19.000 [notice] Bootstrapped 14% (handshake): Handshaking with a relay
Apr 14 16:01:19.000 [notice] Bootstrapped 15% (handshake_done): Handshake with a relay done
Apr 14 16:01:19.000 [notice] Bootstrapped 75% (enough_dirinfo): Loaded enough directory info to build circuits
Apr 14 16:01:19.000 [notice] Bootstrapped 90% (ap_handshake_done): Handshake finished with a relay to build circuits
Apr 14 16:01:19.000 [notice] Bootstrapped 95% (circuit_create): Establishing a Tor circuit
Apr 14 16:01:20.000 [notice] Bootstrapped 100% (done): Done
```

- Check using `telnet 127.0.0.1 9051`

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
○ cs790@cs790:~/Assignments/Assignment_3/tor-coding$ telnet 127.0.0.1 9051
Trying 127.0.0.1...
Connected to 127.0.0.1.
Escape character is '^]'.
█
```

- `ControlPort 9051`: It specifies which port to use for control
- `CookieAuthentication 1`: Tor creates a special file when it runs
ex. `control_auth_cookie`
- Install `stem` and create `python` coding setup

```
cs790@cs790:~/Assignments/Assignment_3$ ls
tor
cs790@cs790:~/Assignments/Assignment_3$ mkdir tor-coding
cs790@cs790:~/Assignments/Assignment_3$ cd tor-coding/
cs790@cs790:~/Assignments/Assignment_3/tor-coding$ ls
cs790@cs790:~/Assignments/Assignment_3/tor-coding$ python3 -m venv venv
cs790@cs790:~/Assignments/Assignment_3/tor-coding$ source venv/bin/activate
(venv) cs790@cs790:~/Assignments/Assignment_3/tor-coding$ pip install stem
Collecting stem
  Downloading stem-1.8.2.tar.gz (2.9 MB)
    2.9/2.9 MB 23.2 MB/s eta 0:00:00
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Preparing metadata (pyproject.toml) ... done
Building wheels for collected packages: stem
  Building wheel for stem (pyproject.toml) ... done
  Created wheel for stem: filename=stem-1.8.2-py3-none-any.whl size=436281 sha256=9839a536051afc96b8c555eb4f60dcd4eaf07406fcb5c6ef7533a4924d393bce
  Stored in directory: /home/cs790/.cache/pip/wheels/91/3d/a8/3f5ed3c98e159d39092654ba746168d5bc58c1eb0d96702f8f
Successfully built stem
Installing collected packages: stem
Successfully installed stem-1.8.2
(venv) cs790@cs790:~/Assignments/Assignment_3/tor-coding$ █
```

- Connecting to tor using stem

```
from stem.control import Controller

with Controller.from_port(port=9051) as controller:
    controller.authenticate()
    print(f"Tor Version: {controller.get_version}")
```

```
configuring_tor.py x
home > cs790 > Assignments > Assignment_3 > tor-coding > configuring_tor.py
1  from stem.control import Controller
2
3  with Controller.from_port(port=9051) as controller:
4      controller.authenticate()
5      print(f"Tor Version: {controller.get_version()}")
6

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
• (venv) cs790@cs790:~/Assignments/Assignment_3/tor-coding$ mv connect_to_tor.py configuring_tor.py
• (venv) cs790@cs790:~/Assignments/Assignment_3/tor-coding$ code configuring_tor.py
• (venv) cs790@cs790:~/Assignments/Assignment_3/tor-coding$ python3 configuring_tor.py
Tor Version: 0.4.9.2-alpha-dev (git-565390ee632e153a)
• (venv) cs790@cs790:~/Assignments/Assignment_3/tor-coding$
```

- Listing all relays

```
from stem.control import Controller

with Controller.from_port(port=9051) as controller:
    controller.authenticate()

    relays = list(controller.get_network_statuses())
    print(f"Found {len(relays)} relays!")
    for i, desc in enumerate(relays[:10]):
        print(f"{i+1}: {desc.fingerprint} ({desc.nickname}")
```

```

configuring_tor.py x
home > cs790 > Assignments > Assignment_3 > tor-coding > configuring_tor.py
1  from stem.control import Controller
2
3  with Controller.from_port(port=9051) as controller:
4      controller.authenticate()
5
6      relays = list(controller.get_network_statuses())
7      print(f"Found {len(relays)} relays!\n")
8      # print(f"Tor Version: {controller.get_version()}")
9      for i, desc in enumerate(relays[:10]):
10         print(f"{i+1}: {desc.fingerprint} ({desc.nickname})")
11
~~~~~
TypeError: object of type 'generator' has no len()
(venv) cs790@cs790:~/Assignments/Assignment_3/tor-coding$ python3 configuring_tor.py
Traceback (most recent call last):
  File "/home/cs790/Assignments/Assignment_3/tor-coding/configuring_tor.py", line 9, in <module>
    for i, desc in enumerate(relays[:10]):
                        ~~~~~~
TypeError: 'generator' object is not subscriptable
(venv) cs790@cs790:~/Assignments/Assignment_3/tor-coding$ python3 configuring_tor.py
Found 8819 relays!

1: 000A10D43011EA4928A35F610405F92B4433B4DC (seele)
2: 000D11C0BF30C48FD81644ED2AA3D3733A5149D1 (hubbabubbaABC)
3: 000F3EB75342BE371F1D8D3FAE90890AEB5664EE (SENDN00SEplz)
4: 0011254CC8444369B20EF11156B8990438221A54 (titamon3)
5: 0011BD2485AD45D984EC4159C88FC066E5E3300E (CalyxInstitute14)
6: 0017A4738B6FFC5E3D0FC7EE380C10FC2160328A (MORDEKAISER)
7: 00194C4FCEB999CF8C7EF1D202AE9BF24D17545 (NipseHussle)
8: 00240ECB2B535AA4C1E1874D744DFA6AF2E5E941 (skylarkRelay)
9: 00251F9F626F0E741030AF54CE019B115D01697D (Quetzalcoatl)
10: 00283B5564E3072DCDDAB31D6EF622DD49BF524F (StarAppsMobley)
(venv) cs790@cs790:~/Assignments/Assignment_3/tor-coding$ python3 configuring_tor.py
Found 8819 relays!

1: 000A10D43011EA4928A35F610405F92B4433B4DC (seele)
2: 000D11C0BF30C48FD81644ED2AA3D3733A5149D1 (hubbabubbaABC)
3: 000F3EB75342BE371F1D8D3FAE90890AEB5664EE (SENDN00SEplz)
4: 0011254CC8444369B20EF11156B8990438221A54 (titamon3)
5: 0011BD2485AD45D984EC4159C88FC066E5E3300E (CalyxInstitute14)
6: 0017A4738B6FFC5E3D0FC7EE380C10FC2160328A (MORDEKAISER)
7: 00194C4FCEB999CF8C7EF1D202AE9BF24D17545 (NipseHussle)
8: 00240ECB2B535AA4C1E1874D744DFA6AF2E5E941 (skylarkRelay)
9: 00251F9F626F0E741030AF54CE019B115D01697D (Quetzalcoatl)
10: 00283B5564E3072DCDDAB31D6EF622DD49BF524F (StarAppsMobley)
(venv) cs790@cs790:~/Assignments/Assignment_3/tor-coding$

```

- Creating 4 hop circuit. Repeatedly try until find good stable relays and then create circuit.

```

import random

# Helper function to pick a relay that has all required
flags
def pick_relays(relays, required_flags):

```

```

    # Filter relays that have *all* the required flags
    options = [r for r in relays if all(flag in r.flags
for flag in required_flags)]
    # Return a random choice from the filtered list, or
    None if no options
    return random.choice(options) if options else None

# Main function to create a 4-hop Tor circuit using Stem
def create_4_hop_circuit(controller):
    circuit_id = -1 # Default circuit ID in case of
failure

    # Get the list of all relays from the currently
    running Tor instance
    relays = list(controller.get_network_statuses())

    # Try up to 15 times to build a valid circuit
    for attempt in range(15):
        print(f"Attempt {attempt + 1}/15")

        try:
            # Pick a suitable guard node with specific
            flags
            guard = pick_relays(relays, ["Guard", "Fast",
"Running", "Stable", "Valid"])

            # Pick two middle relays (don't need
            Guard/Exit flag, but should be fast and stable)
            middle_1 = pick_relays(relays, ["Fast",
"Running", "Stable", "Valid"])
            middle_2 = pick_relays(relays, ["Fast",
"Running", "Stable", "Valid"])

            # Pick an exit node with Exit and other

```

```

reliability flags
    exit_node = pick_relays(relays, ["Exit",
"Fast", "Running", "Stable", "Valid"])

    # If any role couldn't be filled, try again
    if not all([guard, middle_1, middle_2,
exit_node]):
        print("Could not find suitable relays,
retrying...")
        continue

    # Make sure all 4 relays are unique (no
duplicates)
    fps = {r.fingerprint for r in [guard,
middle_1, middle_2, exit_node]}
    if len(fps) < 4:
        print("Duplicate relays detected,
retrying...")
        continue

    # Build the path as a list of fingerprints in
order: Guard → M1 → M2 → Exit
    path = [guard.fingerprint,
middle_1.fingerprint, middle_2.fingerprint,
exit_node.fingerprint]

    # Use Stem to request Tor to build a new
circuit with the selected path
    circuit_id = controller.new_circuit(path=path,
await_build=True)

    # Print out the final selected relays for each
hop

    print("Picked Relays: ")
    print(f"Guard      : {guard.fingerprint}

```

```

({guard.nickname}))")
        print(f"Middle #1 : {middle_1.fingerprint}
({middle_1.nickname}))")
        print(f"Middle #2 : {middle_2.fingerprint}
({middle_2.nickname}))")
        print(f"Exit      : {exit_node.fingerprint}
({exit_node.nickname}))")

        break # Circuit built successfully, break out
of loop

    except Exception as e:
        # Catch and print any error that occurs during
circuit creation
        print(f"Failed to build circuit: {e}")

    return int(circuit_id) # Return the circuit ID (or -1
if failed after 15 tries)

```

```

-----
No Custom 4 hops circuits available
=====
1. Create 4 hop circuit
2. Exit
Choose: 1
-----
Attempt 1/15
Picked Relays:
Guard      : 1D96ECEECD5C62EA18FE95B61B5936447FA1178D (prsv)
Middle #1  : D3E26FE8F609DE911EB433F472266E70FF3013EB (stielmus)
Middle #2  : 2454F02E7434E197E48DDE3A84DA6F77EB41A9D5 (JoeBalogs)
Exit       : 85D5976EAA9EEDC43737209A80678A245145F806 (artikel10ber115)
Circuit id: 12
=====
1. Simple get request through tor default
2. Simple get request through custom tor circuits
3. Close all custom circuits
4. Exit
Choose: █

```

- Using `nyx` tool and check if new circuit is available in list of circuits

```

nyx - cs790 (Linux 6.11.0-21-generic) Tor 0.4.9.2-alpha-dev (new) cpu: 0.2% tor, 8.1% nyx mem: 82 MB (2.1%) pid: 12660 uptime: 00:56
Relaying Disabled, Control Port (cookie): 9051
press 'n' for a new identity

page 2 / 5 - n: menu, p: pause, h: page help, q: quit
Connections (3 outbound, 12 circuit, 2 control)
107.189.8.133:9000 (lu) 09CA1957EC06710440AD2EEA282A348FFD70271E Quetzalcoatl 3 / End
127.0.0.1 --> 107.189.8.133:9000 (lu) Purpose: Conflux_linked, Circuit ID: 4 55.3s (CIRCUIT)
38.244.167.206:443 (us) E2D81758E83CF91BDE8007D4A2866C3BAE8081FE Unnamed 1 / Guard
51.89.81.247:443 (ie) B6F6A56C50B1CFB1869269592383E429758C9A3A Anguiped 2 / Middle
107.189.8.133:9000 (lu) 09CA1957EC06710440AD2EEA282A348FFD70271E Quetzalcoatl 3 / End
127.0.0.1 --> 109.70.100.70:9004 (at) Purpose: Conflux_linked, Circuit ID: 5 55.3s (CIRCUIT)
38.244.167.206:443 (us) E2D81758E83CF91BDE8007D4A2866C3BAE8081FE Unnamed 1 / Guard
89.41.181.222:443 (ro) 009B3CE816364C7C631719927DEFE99452D1A789 dicymb 2 / Middle
109.70.100.70:9004 (at) 106E3C8CF8F5774B9E97732A37DACE99503636 hamster 3 / End
127.0.0.1 --> 109.70.100.70:9004 (at) Purpose: Conflux_linked, Circuit ID: 6 55.3s (CIRCUIT)
176.9.161.22:443 (de) 9EEE861CD8A5FE26A1F276B79323172F5B91082C NlxOS 1 / Guard
87.106.129.242:9001 (de) BAE5A47C968805E60BC051CA58610850C21F96D DontMentionTheWar 2 / Middle
109.70.100.70:9004 (at) 106E3C8CF8F5774B9E97732A37DACE99503636 hamster 3 / End
127.0.0.1 --> 178.33.45.159:8080 (fr) Purpose: Hs.vanguards, Circuit ID: 10 52.3s (CIRCUIT)
38.244.167.206:443 (us) E2D81758E83CF91BDE8007D4A2866C3BAE8081FE Unnamed 1 / Guard
217.168.114.102:9001 (es) 4C122DFAED8BD43AD638B1045AF53A9708F17695 RelayUll 2 / Middle
178.33.45.159:8080 (fr) A07D56AC8FF3B7539B314222B658B1190AB66514 htx02 3 / End
127.0.0.1 --> 185.220.100.247:9000 (de) Purpose: Hs.vanguards, Circuit ID: 9 53.3s (CIRCUIT)
176.9.161.22:443 (de) 9EEE861CD8A5FE26A1F276B79323172F5B91082C NlxOS 1 / Guard
45.154.98.14:143 (nl) 02584620864070FEBA374931A5E5263454E8E609 RDPdotSH 2 / Middle
185.220.100.247:9000 (de) 5E52BEA22130598F200D05C428B66709C24E8F67 relayondagon 3 / End
127.0.0.1 --> 185.220.101.25:9001 (de) Purpose: General, Circuit ID: 7 55.3s (CIRCUIT)
176.9.161.22:443 (de) 9EEE861CD8A5FE26A1F276B79323172F5B91082C NlxOS 1 / Guard
185.177.229.228:8080 (au) 5F276A6F7AA74AFB2AF100EADA28C7A6F48BA58F Planetclaire61 2 / Middle
185.220.101.25:9001 (de) F447CACAC53D4100B1DE4640140B4CFC0A1FC5C artikel10ber97 3 / End
127.0.0.1 --> 185.220.101.29:9003 (de) Purpose: General, Circuit ID: 12 42.6s (CIRCUIT)
51.77.132.82:9600 (fr) 1096CEECDC5C62EA18FE9586185936447FA1178D prsv 1 / Guard
109.70.100.66:9001 (at) D3E26FE8F609DE911EB433F472266E78FF3013EB stielmus 2 / Middle
87.106.65.173:443 (gb) 2454F02E7434E197E480DE3A84DA6F77EB41A905 JoeBalogos 3 / Middle
185.220.101.29:9003 (de) 85D5976EAA9EEDC43737209A80678A245145F806 artikel10ber115 4 / End

```

Simplified using Makefile

- Run following commands

```
make clean
```

```
make
```

```

cs790@cs790: ~/Assignments/Assignment_3/Submission/Task_1
cs790@cs790:~/Assignments/Assignment_3/Submission/Task_1$ make clean
rm -rf venv
cs790@cs790:~/Assignments/Assignment_3/Submission/Task_1$ make
venv/bin/pip install --upgrade pip
Requirement already satisfied: pip in ./venv/lib/python3.12/site-packages (24.0)
Collecting pip
  Using cached pip-25.0.1-py3-none-any.whl.metadata (3.7 kB)
Using cached pip-25.0.1-py3-none-any.whl (1.8 MB)
Installing collected packages: pip
  Attempting uninstall: pip
    Found existing installation: pip 24.0
    Uninstalling pip-24.0:
      Successfully uninstalled pip-24.0
Successfully installed pip-25.0.1
venv/bin/pip install -r requirements.txt
Collecting beautifulsoup4==4.13.3 (from -r requirements.txt (line 1))
  Using cached beautifulsoup4-4.13.3-py3-none-any.whl.metadata (3.8 kB)
Collecting bs4==0.0.2 (from -r requirements.txt (line 2))
  Using cached bs4-0.0.2-py2.py3-none-any.whl.metadata (411 bytes)
Collecting certifi==2025.1.31 (from -r requirements.txt (line 3))
  Using cached certifi-2025.1.31-py3-none-any.whl.metadata (2.5 kB)
Collecting charset-normalizer==3.4.1 (from -r requirements.txt (line 4))
  Using cached charset_normalizer-3.4.1-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (35 kB)
Collecting idna==3.10 (from -r requirements.txt (line 5))
  Using cached idna-3.10-py3-none-any.whl.metadata (10 kB)
Collecting PySocks==1.7.1 (from -r requirements.txt (line 6))
  Using cached PySocks-1.7.1-py3-none-any.whl.metadata (13 kB)
Collecting requests==2.32.3 (from -r requirements.txt (line 7))
  Using cached requests-2.32.3-py3-none-any.whl.metadata (4.6 kB)
Collecting soupsteve==2.6 (from -r requirements.txt (line 8))
  Using cached soupsteve-2.6-py3-none-any.whl.metadata (4.6 kB)
Collecting stem==1.8.2 (from -r requirements.txt (line 9))
  Using cached stem-1.8.2-py3-none-any.whl
Collecting typing_extensions==4.13.2 (from -r requirements.txt (line 10))
  Using cached typing_extensions-4.13.2-py3-none-any.whl.metadata (3.0 kB)
Collecting urllib3==2.4.0 (from -r requirements.txt (line 11))
  Using cached urllib3-2.4.0-py3-none-any.whl.metadata (6.5 kB)
Using cached beautifulsoup4-4.13.3-py3-none-any.whl (186 kB)
Using cached bs4-0.0.2-py2.py3-none-any.whl (1.2 kB)
Using cached certifi-2025.1.31-py3-none-any.whl (166 kB)
Using cached charset-normalizer-3.4.1-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (145 kB)
Using cached idna-3.10-py3-none-any.whl (70 kB)
Using cached PySocks-1.7.1-py3-none-any.whl (16 kB)
Using cached requests-2.32.3-py3-none-any.whl (64 kB)
Using cached soupsteve-2.6-py3-none-any.whl (36 kB)

```

```

venv/bin/python task_1.py
-----
No Custom 4 hops circuits available
=====
1. Create 4 hop circuit
2. Exit
Choose: 

```

3. Finally, you need to show the screenshot with the 4-hop circuit and the TCP connection that goes through that. To test it, you can access any site using the Tor circuit. You can connect to the Tor control port directly or using the `nyx` program.

- Now, let's just use default circuit and send request to

`https://facebook.com`

```
=====
1. Simple get request through tor default
2. Simple get request through custom tor circuits
3. Close all custom circuits
4. Exit
Choose: 1
-----
Enter url: https://facebook.com
200
<!DOCTYPE html>
<html lang="nl" id="facebook" class="no_js">
<head><meta charset="utf-8" /><meta name="referrer" content="default" id="meta referrer" /><script nonce="bjsbgBTAS">function envFlush(a){func
n b(b){for(var c in a)b[c]=a[c]}window.requireLazy?window.requireLazy(["Env"],b):(window.Env=window.Env||{}),b(window.Env))}envFlush({"useTrust
Types":false,"isTrustedTypesReportOnly":false,"ajaxpipe_token":"AXg8ppLLDnxuj43Rl4","stack_trace_limit":30,"timesliceBufferSize":5000,"show_i
ariant_
=====
1. Simple get request through tor default
2. Simple get request through custom tor circuits
3. Close all custom circuits
4. Exit
Choose: |
```

```
nyx - cs790 (Linux 6.11.0-21-generic) Tor 0.4.9.2-alpha-dev (new)
Relaying Disabled, Control Port (cookie): 9051
cpu: 0.4% tor, 1.1% nyx mem: 82 MB (2.1%) pid: 12660 uptime: 02:39
press 'n' for a new identity

page 1 / 5 - m: menu, p: pause, h: page help, q: quit
Bandwidth (limit: 1 GB/s, burst: 1 GB/s):
Download (0.0 B/sec - avg: 961.5 B/sec, total: 151.5 KB):
30 KB
20 KB
10 KB
0 B
10s 20 30 40 50 1m 1.1 1.3
Upload (0.0 B/sec - avg: 480.7 B/sec, total: 75.7 KB):
6 KB
4 KB
2 KB
0 B
10s 20 30 40 50 1m 1.1 1.3

Events (TOR/NYX NOTICE-ERR, STREAM):
17:45:33 [STREAM] 10 CLOSED 1 157.240.247.35:443 REASON=DONE CLIENT_PROTOCOL=SOCKS5 NYM_EPOCH=0 SESSION_GROUP=-4 ISO_FIELDS=SOCKS_USERNAME,SOCKS_PASSWORD,CLIENTADDR,SESSION_GROUP,NYM_EPOCH
17:45:33 [STREAM] 11 CLOSED 1 157.240.247.35:443 REASON=DONE CLIENT_PROTOCOL=SOCKS5 NYM_EPOCH=0 SESSION_GROUP=-4 ISO_FIELDS=SOCKS_USERNAME,SOCKS_PASSWORD,CLIENTADDR,SESSION_GROUP,NYM_EPOCH
17:45:31 [STREAM] 11 SUCCEEDED 1 157.240.247.35:443 CLIENT_PROTOCOL=SOCKS5 NYM_EPOCH=0 SESSION_GROUP=-4 ISO_FIELDS=SOCKS_USERNAME,SOCKS_PASSWORD,CLIENTADDR,SESSION_GROUP,NYM_EPOCH
17:45:31 [STREAM] 11 REMAP 1 157.240.247.35:443 SOURCE=EXIT CLIENT_PROTOCOL=SOCKS5 NYM_EPOCH=0 SESSION_GROUP=-4 ISO_FIELDS=SOCKS_USERNAME,SOCKS_PASSWORD,CLIENTADDR,SESSION_GROUP,NYM_EPOCH
17:45:31 [STREAM] 11 SENTCONNECT 1 www.facebook.com:443 CLIENT_PROTOCOL=SOCKS5 NYM_EPOCH=0 SESSION_GROUP=-4 ISO_FIELDS=SOCKS_USERNAME,SOCKS_PASSWORD,CLIENTADDR,SESSION_GROUP,NYM_EPOCH
17:45:31 [STREAM] 11 NEW 0 www.facebook.com:443 SOURCE_ADDR=127.0.0.1:45334 PURPOSE=USER CLIENT_PROTOCOL=SOCKS5 NYM_EPOCH=0 SESSION_GROUP=-4
ISO_FIELDS=SOCKS_USERNAME,SOCKS_PASSWORD,CLIENTADDR,SESSION_GROUP,NYM_EPOCH
17:45:30 [STREAM] 10 SUCCEEDED 1 157.240.247.35:443 CLIENT_PROTOCOL=SOCKS5 NYM_EPOCH=0 SESSION_GROUP=-4 ISO_FIELDS=SOCKS_USERNAME,SOCKS_PASSWORD,CLIENTADDR,SESSION_GROUP,NYM_EPOCH
17:45:30 [STREAM] 10 REMAP 1 157.240.247.35:443 SOURCE=EXIT CLIENT_PROTOCOL=SOCKS5 NYM_EPOCH=0 SESSION_GROUP=-4 ISO_FIELDS=SOCKS_USERNAME,SOCKS_PASSWORD,CLIENTADDR,SESSION_GROUP,NYM_EPOCH
17:45:30 [STREAM] 10 SENTCONNECT 1 facebook.com:443 CLIENT_PROTOCOL=SOCKS5 NYM_EPOCH=0 SESSION_GROUP=-4 ISO_FIELDS=SOCKS_USERNAME,SOCKS_PASSWORD,CLIENTADDR,SESSION_GROUP,NYM_EPOCH
17:45:30 [STREAM] 10 NEW 0 facebook.com:443 SOURCE_ADDR=127.0.0.1:45330 PURPOSE=USER CLIENT_PROTOCOL=SOCKS5 NYM_EPOCH=0 SESSION_GROUP=-4
ISO_FIELDS=SOCKS_USERNAME,SOCKS_PASSWORD,CLIENTADDR,SESSION_GROUP,NYM_EPOCH
17:43:44 [NYX_NOTICE] No nyxrc loaded, using defaults. You can customize nyx by placing a configuration file at /home/cs790/.nyx/config (see https://nyx.torproject.org/nyxrc.sample for its
options).
```

- Let's use our custom circuit 12

```
=====
1. Simple get request through tor default
2. Simple get request through custom tor circuits
3. Close all custom circuits
4. Exit
Choose: 2
-----
Enter url: https://facebook.com
-----
Available custom 4 hop circuits
1. Circuit id: 12
-----
Enter Circuit ID: 12
Making request to https://facebook.com
Attaching stream 12 to circuit 12
Attaching stream 13 to circuit 12
Request completed.
=====
1. Simple get request through tor default
2. Simple get request through custom tor circuits
3. Close all custom circuits
4. Exit
Choose: 
```



- Following is the code for attaching stream to newly created circuit

```
import threading
import requests
from stem.control import EventType

# Sends a GET request using Tor's *default* circuit
def tor_default_get_request(url):
    # Makes a normal request over Tor using your default
    proxy settings
```

```
res = requests.get(url, proxies=proxies)
print(res.status_code)
print(res.text[:500]) # Print first 500 characters of
the response

# Sends a GET request using a *custom Tor circuit* built
with Stem
def tor_custom_get_request(controller, url, circ):
    # Tell Tor not to automatically attach new streams to
a circuit
    controller.set_conf('__LeaveStreamsUnattached', '1')

    # This event helps us know when the stream is
successfully attached
    stream_attached = threading.Event()

    # Event listener: called whenever a new stream is
created
    def attach_stream(stream):
        if stream.status == 'NEW':
            print(f"Attaching stream {stream.id} to
circuit {circ}")
            try:
                # Attach this stream manually to our
chosen circuit
                controller.attach_stream(stream.id, circ)
                # Signal that the stream has been attached
successfully
                stream_attached.set()
            except Exception as e:
                print(f"Failed to attach stream: {e}")

    # Register the event listener to catch STREAM events
    controller.add_event_listener(attach_stream,
EventType.STREAM)
```

```
try:
    print(f"Making request to {url}")

    # Start the HTTP request in a separate thread (so
    # we don't block the stream attachment)
    response_thread = threading.Thread(
        target=lambda: requests.get(url,
proxies=proxies)
    )
    response_thread.start()

    # Wait for up to 10 seconds for the stream to be
    # attached to our circuit
    stream_attached.wait(timeout=10)

    # Wait for the request to complete
    response_thread.join()
    print("Request completed.")

finally:
    # Remove listener and restore default Tor behavior
    controller.remove_event_listener(attach_stream)
    controller.reset_conf('__LeaveStreamsUnattached')
```

Full Code

```
import random
import threading
import requests
from stem.control import Controller, EventType

proxies = {
```

```

    'http': 'socks5h://127.0.0.1:9050',
    'https': 'socks5h://127.0.0.1:9050'
}

def pick_relays(relays, required_flags):
    options = [r for r in relays if all(flag in r.flags
for flag in required_flags)]
    return random.choice(options) if options else None

def create_4_hop_circuit(controller):
    circuit_id = -1
    relays = list(controller.get_network_statuses())

    for attempt in range(15):
        print(f"Attempt {attempt + 1}/15")

        try:
            guard = pick_relays(relays, ["Guard", "Fast",
"Running", "Stable", "Valid"])
            middle_1 = pick_relays(relays, ["Fast",
"Running", "Stable", "Valid"])
            middle_2 = pick_relays(relays, ["Fast",
"Running", "Stable", "Valid"])
            exit_node = pick_relays(relays, ["Exit",
"Fast", "Running", "Stable", "Valid"])

            if not all([guard, middle_1, middle_2,
exit_node]):
                print("Could not find suitable relays,
retrying...")
                continue

            fps = {r.fingerprint for r in [guard,
middle_1, middle_2, exit_node]}
            if len(fps) < 4:

```

```

        print("Duplicate relays detected,
retrying...")
        continue

    path = [guard.fingerprint,
middle_1.fingerprint, middle_2.fingerprint,
exit_node.fingerprint]
    circuit_id = controller.new_circuit(path=path,
await_build=True)

    print("Picked Relays: ")
    print(f"Guard      : {guard.fingerprint}
({guard.nickname})")
    print(f"Middle #1 : {middle_1.fingerprint}
({middle_1.nickname})")
    print(f"Middle #2 : {middle_2.fingerprint}
({middle_2.nickname})")
    print(f"Exit      : {exit_node.fingerprint}
({exit_node.nickname})")
    break
except Exception as e:
    print(f"Failed to build circuit: {e}")
return int(circuit_id)

def close_circuit(controller, circuit_id):
    controller.close_circuit(circuit_id)
    print(f"Closed circuit {circuit_id}")

def get_available_custom_circuits(controller):
    circuits = controller.get_circuits()
    circuits_with_4_hops = []
    for circ in circuits:
        if circ.status != 'BUILT':

```

```

        continue

    if len(circ.path) == 4:
        circuits_with_4_hops.append(int(circ.id))
    return circuits_with_4_hops

def close_all_custom_circuits(controller):
    circuits_with_4_hops =
get_available_custom_circuits(controller)
    for circ in circuits_with_4_hops:
        close_circuit(controller=controller,
circuit_id=circ)

def tor_default_get_request(url):
    res = requests.get(url, proxies=proxies)
    print(res.status_code)
    print(res.text[:500])

def tor_custom_get_request(controller, url, circ):
    controller.set_conf('__LeaveStreamsUnattached', '1')
    stream_attached = threading.Event()

    def attach_stream(stream):
        if stream.status == 'NEW':
            print(f"Attaching stream {stream.id} to
circuit {circ}")
            try:
                controller.attach_stream(stream.id, circ)
                stream_attached.set()
            except Exception as e:
                print(f"Failed to attach stream: {e}")

    controller.add_event_listener(attach_stream,
EventType.STREAM)

```

```

try:
    print(f"Making request to {url}")
    response_thread = threading.Thread(target=lambda:
requests.get(url, proxies=proxies))
    response_thread.start()

    stream_attached.wait(timeout=10)

    response_thread.join()
    print("Request completed.")

finally:
    controller.remove_event_listener(attach_stream)
    controller.reset_conf('__LeaveStreamsUnattached')

with Controller.from_port(port=9051) as controller:
    controller.authenticate()

is_quit = False
try:
    while not is_quit:
        circuits_with_4_hops =
get_available_custom_circuits(controller)
        if len(circuits_with_4_hops) == 0:
            print("-----")
            print("No Custom 4 hops circuits
available")

            print("=====")
            print(f"1. Create 4 hop circuit")
            print(f"2. Exit")
            choice = int(input("Choose: "))
            print("-----")
            if choice == 1:

```

```

        circuit_id =
create_4_hop_circuit(controller)
        if circuit_id > 0:
            print(f"Circuit id: {circuit_id}")
        else:
            print(f"Faield to create circuit!
Try again...")
        else:
            is_quit = True

    else:

print("=====")
        print(f"1. Simple get request through tor
default")
        print(f"2. Simple get request through
custom tor circuits")
        print(f"3. Close all custom circuits")
        print(f"4. Exit")
        choice = int(input("Choose: "))
        print("-----")
        if choice < 3:
            url = input("Enter url: ")
            if choice == 1:
                tor_default_get_request(url)
            elif choice == 2:
                print("-" * 20)
                print("Available custom 4 hop
circuits")

                for i, circ in
enumerate(circuits_with_4_hops):
                    print(f"{i+1}. Circuit id:
{circ}")

                print("-" * 20)
                selected_circ = input("Enter

```

```
Circuit ID: ")

tor_custom_get_request(controller=controller, url=url,
circ=selected_circ)

        elif choice == 3:

close_all_custom_circuits(controller=controller)
        else:
            is_quit = True

    except Exception as e:
        print(f"Error: {e}")

    finally:
        controller.close()

print("End!")
```